

SwiftEx Courier System



Submitted By:

Muhammad Arslan	2024-CS-632
Afaaq Anwar	2024-CS-603
Ahsan Shakeel	2024-CS-638
Taimoor Ali	2024-CS-609

Submitted To:

Sir Ali Raza

Department of Computer Science
University of Engineering and Technology Lahore, New Campus

Table of Contents

Executive Summary	3
Problem Statement	4
System Architecture	4
Overall Architecture	4
Component Breakdown	4
Data Structures Used	5
1. Vector (Dynamic Array)	5
2. LinkedList	5
3. Queue (FIFO)	5
4. MinHeap (Priority Queue)	5
5. Graph (Adjacency List)	5
6. HashTable (Separate Chaining)	6
7. Stack (LIFO)	6
Algorithm Implementation	6
1. Dijkstra's Algorithm (Shortest Path)	6
2. K-Shortest Paths (Yen's Algorithm - Simplified)	6
3. Multi-Criteria Parcel Sorting	6
4. Rider Assignment Algorithm	7
Module Design	7
Module 1: Intelligent Parcel Sorting	7
Module 2: Parcel Routing	7
Module 3: Parcel Tracking	8
Module 4: Courier Operations Engine	8
System Features	9
Core Features	9
User Interface	10
Complexity Analysis	10
Time Complexity Summary	10
Performance Optimizations	10
Code Structure	11
File Organization	11

Key Classes and Structures	11
Testing & Validation	11
Test Scenarios	11
Validation Features	12
Data Structure Justification.....	12
Why Custom Implementations?	12
Design Decisions	12
Class Diagram	13
Flowcharts	14
System Architecture.....	14
Operation Menu	14
Parcel Lifecycle	14
Routing.....	15
Parcel Tracking	16
Data Structures.....	16
Authentication	17
Report Statistics	18
Conclusion.....	19
Future Enhancements	19
Learning Outcomes.....	19
References	19

Executive Summary

The **SwiftEx Courier System** is an intelligent logistics engine designed to manage parcel sorting, routing, tracking, and delivery operations efficiently. The system implements advanced data structures and algorithms to handle real-world courier operations including priority-based sorting, shortest path routing, real-time tracking, and comprehensive operation management.

Key Achievements: - Implementation of 7+ custom data structures - Multi-criteria parcel sorting algorithm - Dijkstra's algorithm for shortest path routing - K-shortest paths algorithm for alternative routes - Complete parcel lifecycle tracking - Operation logging with undo/replay functionality

Problem Statement

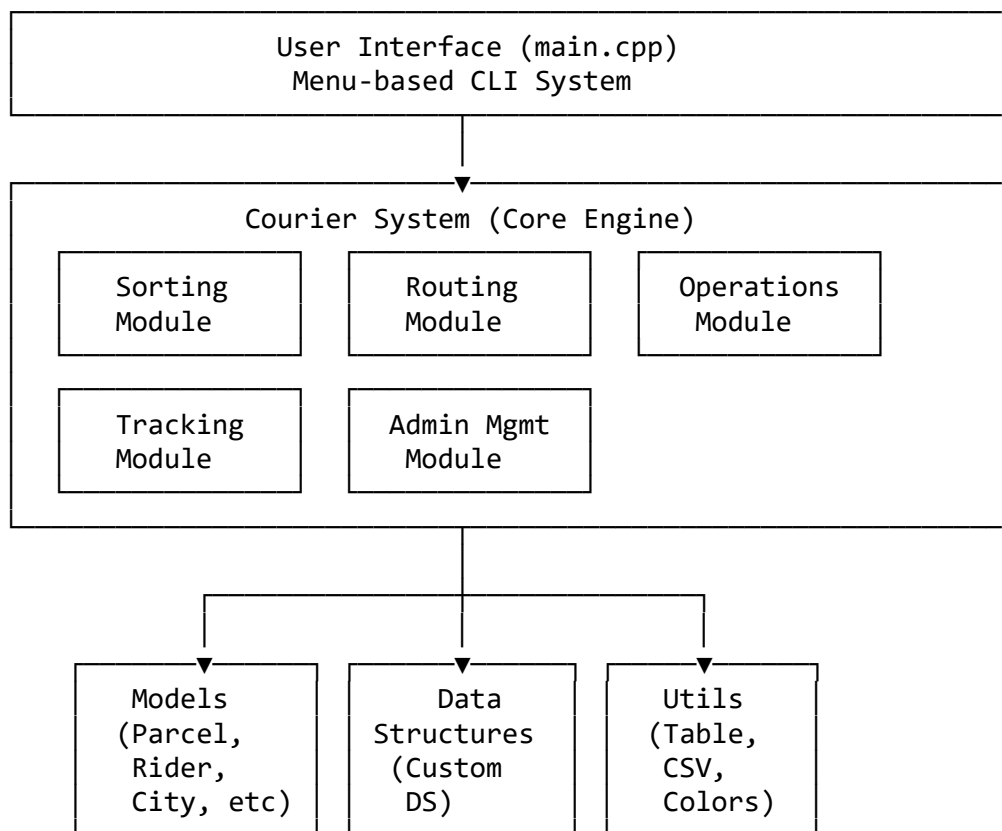
SwiftEx Logistics faces challenges managing thousands of parcels daily with varying priorities, weights, destinations, and delivery requirements. The system needs to:

1. **Intelligently sort parcels** based on priority, weight, and destination
 2. **Calculate optimal routes** while handling blocked/overloaded paths
 3. **Track parcels** through their complete lifecycle
 4. **Manage operations** including rider assignment, queue management, and missing parcel detection
-

System Architecture

Overall Architecture

The system follows a modular architecture with clear separation of concerns:



Component Breakdown

1. **CourierSystem**: Main class orchestrating all operations
2. **Models**: Data structures (Parcel, Rider, City, Admin)
3. **DataStructures**: Custom implementations (Vector, Graph, HashTable, etc.)

- 4. **Utils:** Helper utilities (Table printing, CSV handling, colors)
-

Data Structures Used

1. Vector (Dynamic Array)

- **Purpose:** Storage for parcels, riders, cities, and general collections
- **Time Complexity:**
 - Access: $O(1)$
 - Insertion: $O(1)$ amortized
 - Deletion: $O(n)$
- **Justification:** Efficient random access and iteration needed for displaying lists

2. LinkedList

- **Purpose:**
 - Graph adjacency lists
 - Parcel history tracking
 - Hash table
- **Time Complexity:**
 - Insertion: $O(1)$
 - Search: $O(n)$
- **Justification:** Dynamic size, efficient insertion for history tracking

3. Queue (FIFO)

- **Purpose:**
 - Pickup queue (parcels awaiting processing)
 - Transit queue (parcels in transit)
- **Time Complexity:**
 - Enqueue: $O(1)$
 - Dequeue: $O(1)$
- **Justification:** Maintains order for parcel processing pipeline

4. MinHeap (Priority Queue)

- **Purpose:** Warehouse queue with priority-based sorting
- **Time Complexity:**
 - Insert: $O(\log n)$
 - Extract Min: $O(\log n)$
- **Justification:** Efficient priority-based processing for warehouse operations

5. Graph (Adjacency List)

- **Purpose:** City network representation for routing
- **Time Complexity:**

- Add Edge: $O(1)$
- Get Neighbors: $O(1)$
- Traversal: $O(V + E)$
- **Justification:** Efficient representation of city network for pathfinding

6. HashTable (Separate Chaining)

- **Purpose:** $O(1)$ parcel lookup by tracking ID
- **Time Complexity:**
 - Insert: $O(1)$ average, $O(n)$ worst case
 - Lookup: $O(1)$ average, $O(n)$ worst case
- **Justification:** Fast parcel retrieval essential for tracking operations

7. Stack (LIFO)

- **Purpose:** Operation history for undo functionality
- **Time Complexity:**
 - Push: $O(1)$
 - Pop: $O(1)$
- **Justification:** Natural data structure for undo operations (LIFO order)

Algorithm Implementation

1. Dijkstra's Algorithm (Shortest Path)

- **Purpose:** Find shortest route between two cities
- **Time Complexity:** $O((V + E) \log V)$ where V = vertices, E = edges
- **Space Complexity:** $O(V)$
- **Implementation:** Modified to handle blocked edges and overloaded paths
- **Key Features:**
 - Avoids blocked routes when enabled
 - Handles overloaded paths
 - Returns path as vector of city IDs

2. K-Shortest Paths (Yen's Algorithm - Simplified)

- **Purpose:** Find k alternative routes between cities
- **Time Complexity:** $O(k * V * (E + V \log V))$
- **Space Complexity:** $O(k * V)$
- **Implementation:** Temporarily blocks edges from previous paths to find alternatives

3. Multi-Criteria Parcel Sorting

- **Purpose:** Sort parcels by priority, weight, destination
- **Time Complexity:** $O(n \log n)$ for sorting
- **Criteria Order:**

1. Priority (Overnight > Two-Day > Normal)
2. Fragile status (fragile parcels prioritized)
3. Weight category (Light < Medium < Heavy)
4. Destination city ID
5. Tracking ID (FIFO tiebreaker)

4. Rider Assignment Algorithm

- **Purpose:** Assign parcels to available riders
 - **Strategy:**
 1. Find rider in same city with sufficient capacity
 2. Find rider in adjacent city if none available
 3. Find any available rider (will move to pickup location)
 - **Time Complexity:** $O(R * E)$ where R = riders, E = edges per city
-

Module Design

Module 1: Intelligent Parcel Sorting

Features:

- Add/remove parcels with real-time updates
- Multi-criteria sorting (priority, weight, destination)
- Fragile parcel handling - Category-based filtering

Key Functions:

- `addParcel()`: Add new parcel to system
- `removeParcel()`: Real-time withdrawal
- `sortParcelsByPriority()`: Priority-based sorting
- `sortParcelsByWeight()`: Weight-based sorting
- `sortParcelsByDestination()`: Destination-based sorting
- `markParcelAsFragile()`: Mark fragile parcels

Module 2: Parcel Routing

Features:

- Shortest path calculation
- Alternative route finding
- Blocked route management

- Overloaded path handling
- Dynamic route recalculation

Key Functions:

- `calculateShortestPath()`: Dijkstra's algorithm implementation
- `displayAlternativeRoutes()`: K-shortest paths
- `blockRoute()` / `unblockRoute()`: Route management
- `markPathAsOverloaded()`: Capacity tracking
- `recalculateActiveRoutes()`: Update routes for in-transit parcels

Module 3: Parcel Tracking

Features:

- Complete lifecycle tracking
- Timestamped event history
- Current location tracking
- Route visualization
- Delivery attempt tracking

Key Functions:

- `trackParcel()`: Display complete parcel information
- `recordDispatch()`: Record dispatch events
- `recordLoading()` / `recordUnloading()`: Location updates
- `recordDeliveryAttempt()`: Track delivery attempts
- `returnToSender()`: Handle returns

Module 4: Courier Operations Engine

Features:

- Three-stage queue system (Pickup → Warehouse → Transit)
- Rider assignment with capacity tracking

- Missing parcel detection
- Operation logging
- Undo/replay functionality

Key Functions:

- `processNextParcel()`: Move from pickup to warehouse
 - `assignRider()`: Assign parcel to available rider
 - `completeDelivery()`: Mark parcel as delivered
 - `detectMissingParcels()`: Identify misplaced parcels
 - `undoLastOperation()`: Revert last operation
 - `loadOperationLog()` / `saveOperationLog()`: Log management
-

System Features

Core Features

1. Parcel Management

- Add parcels with priority and weight
- Real-time parcel withdrawal
- View parcels by various criteria
- Mark parcels as fragile

2. Routing & Navigation

- View all cities
- Find shortest routes
- Find alternative routes
- Block/unblock routes
- Handle overloaded paths

3. Operations

- Process orders (Pickup → Warehouse)
- Assign riders (Warehouse → Transit)
- Complete deliveries (Transit → Delivered)
- Record dispatch, loading, unloading events
- Return to sender functionality

4. Rider Management

- Add riders with capacity
- View rider loads

- Track rider assignments
5. **System & Reports**
 - Operation logging
 - Undo operations
 - Delivery statistics
 - Rider statistics
 - Data persistence
 6. **Admin Management**
 - Secure authentication
 - Add/remove admins
 - Super admin protection

User Interface

- Menu-based CLI system
- Color-coded output
- Unicode box-drawing characters for tables
- Intuitive navigation
- Password masking for security

Complexity Analysis

Time Complexity Summary

Operation	Time Complexity	Space Complexity
Add Parcel	$O(1)$	$O(1)$
Find Parcel (by ID)	$O(1)$ avg	$O(1)$
Remove Parcel	$O(n)$	$O(1)$
Calculate Shortest Path	$O((V+E) \log V)$	$O(V)$
K-Shortest Paths	$O(kV/(E+V \log V))$	$O(k*V)$
Sort Parcels	$O(n \log n)$	$O(n)$
Assign Rider	$O(R*E)$	$O(1)$
Undo Operation	$O(n)$	$O(1)$

Where: - n = number of parcels - V = number of cities (vertices) - E = number of routes (edges) - R = number of riders - k = number of alternative paths requested

Performance Optimizations

1. **HashTable for Parcel Lookup:** Reduces lookup time from $O(n)$ to $O(1)$ average case
2. **MinHeap for Warehouse Queue:** Efficient priority-based processing
3. **Graph Adjacency List:** Space-efficient for sparse networks
4. **Queue-based Pipeline:** Maintains order without sorting overhead

Code Structure

File Organization

Courier Service - Final/

— main.cpp	# Entry point, menu system
— CourierSystem.h	# Main system class declaration
— CourierSystem.cpp	# Main system class implementation
— Models.h	# Data models (Parcel, Rider, City, etc.)
— DataStructures.h	# Custom data structure implementations
— Utils.h	# Utility functions (Table, CSV, Colors)
— cities.csv	# City data
— routes.csv	# Route data
— parcels.csv	# Parcel data
— riders.csv	# Rider data
— admins.csv	# Admin data
— compile.bat	# Compilation script

Key Classes and Structures

1. **CourierSystem**: Main orchestrator class
2. **Parcel**: Parcel data model with priority and status
3. **Rider**: Rider data model with capacity tracking
4. **City**: City data model
5. **Admin**: Admin authentication model
6. **Table**: Formatted table printing utility
7. **CSVUtils**: File I/O utilities

Testing & Validation

Test Scenarios

1. **Parcel Management**
 - Add parcels with different priorities
 - Remove parcels at various stages
 - Sort parcels by different criteria
2. **Routing**
 - Calculate shortest paths between cities
 - Find alternative routes
 - Handle blocked routes
 - Test route recalculation
3. **Operations**
 - Process parcels through pipeline
 - Assign riders with capacity constraints

- Complete deliveries
 - Test undo functionality
4. **Edge Cases**
- No available riders
 - All routes blocked
 - Missing parcels
 - Maximum capacity scenarios

Validation Features

- Input validation for all operations
 - Error handling and user feedback
 - Data persistence (CSV files)
 - Operation logging for audit trail
-

Data Structure Justification

Why Custom Implementations?

All data structures are custom-implemented to:

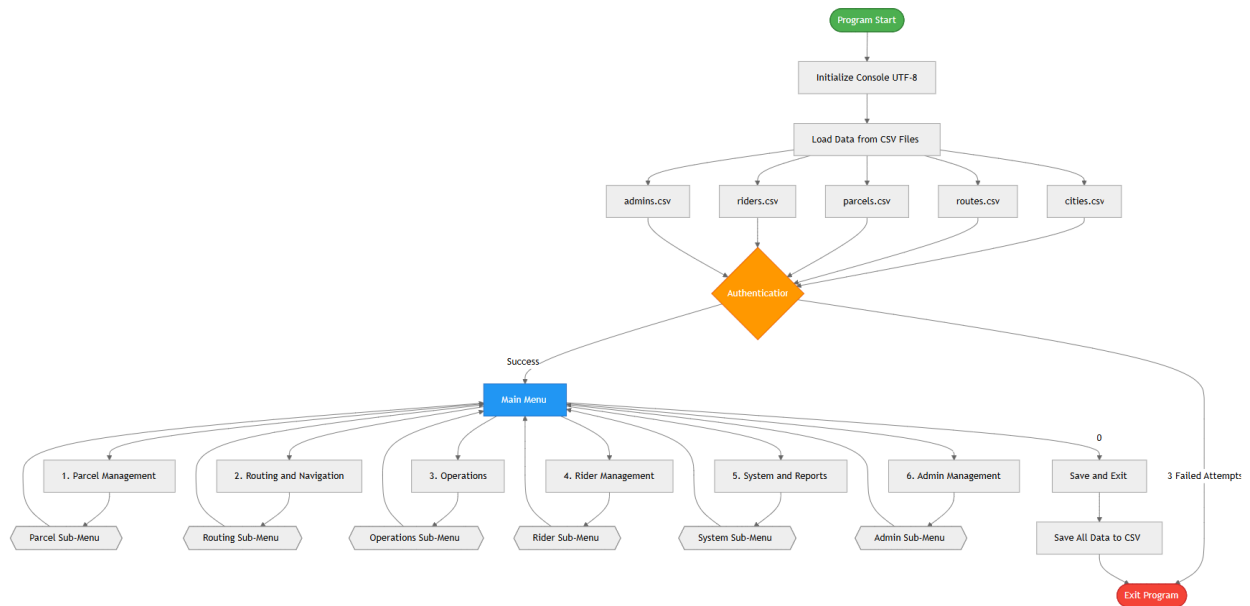
1. Demonstrate understanding of DS&A concepts
2. Avoid STL dependencies (as per requirements)
3. Provide full control over implementation
4. Optimize for specific use cases

Design Decisions

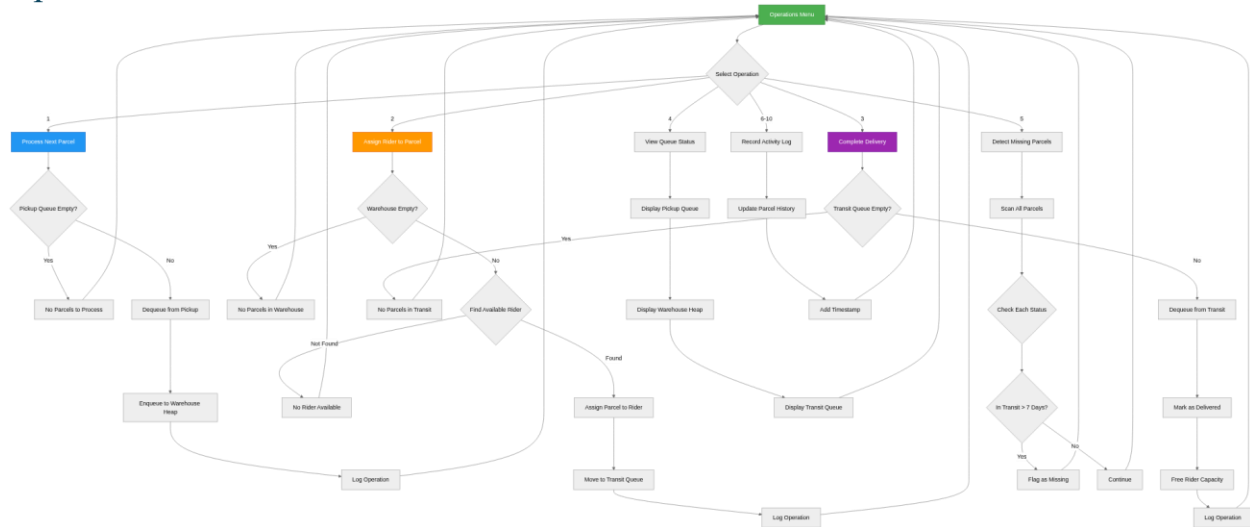
1. **HashTable over Vector for Parcel Lookup**
 - Vector: $O(n)$ lookup time
 - HashTable: $O(1)$ average case
 - Justification: Frequent parcel lookups justify hash table overhead
2. **MinHeap over Vector for Warehouse Queue**
 - Vector: $O(n \log n)$ sorting each time
 - MinHeap: $O(\log n)$ insertion, $O(\log n)$ extraction
 - Justification: Priority-based processing requires efficient priority queue
3. **Graph Adjacency List over Adjacency Matrix**
 - Matrix: $O(V^2)$ space
 - Adjacency List: $O(V + E)$ space
 - Justification: Sparse city network (fewer edges than V^2)
4. **Queue for Pickup/Transit Stages**
 - Maintains FIFO order

Flowcharts

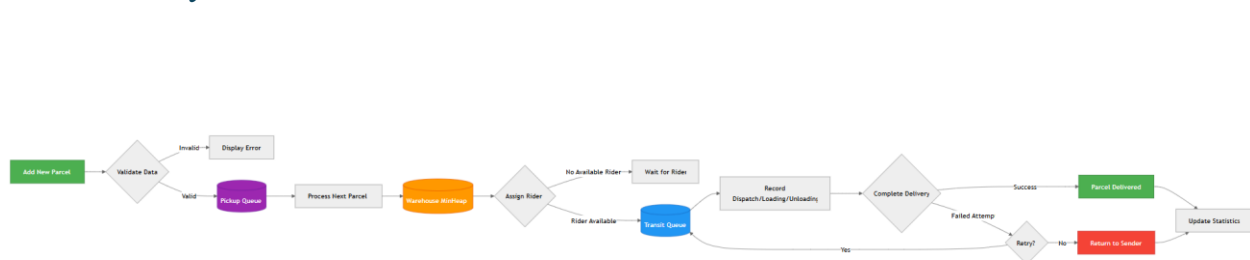
System Architecture



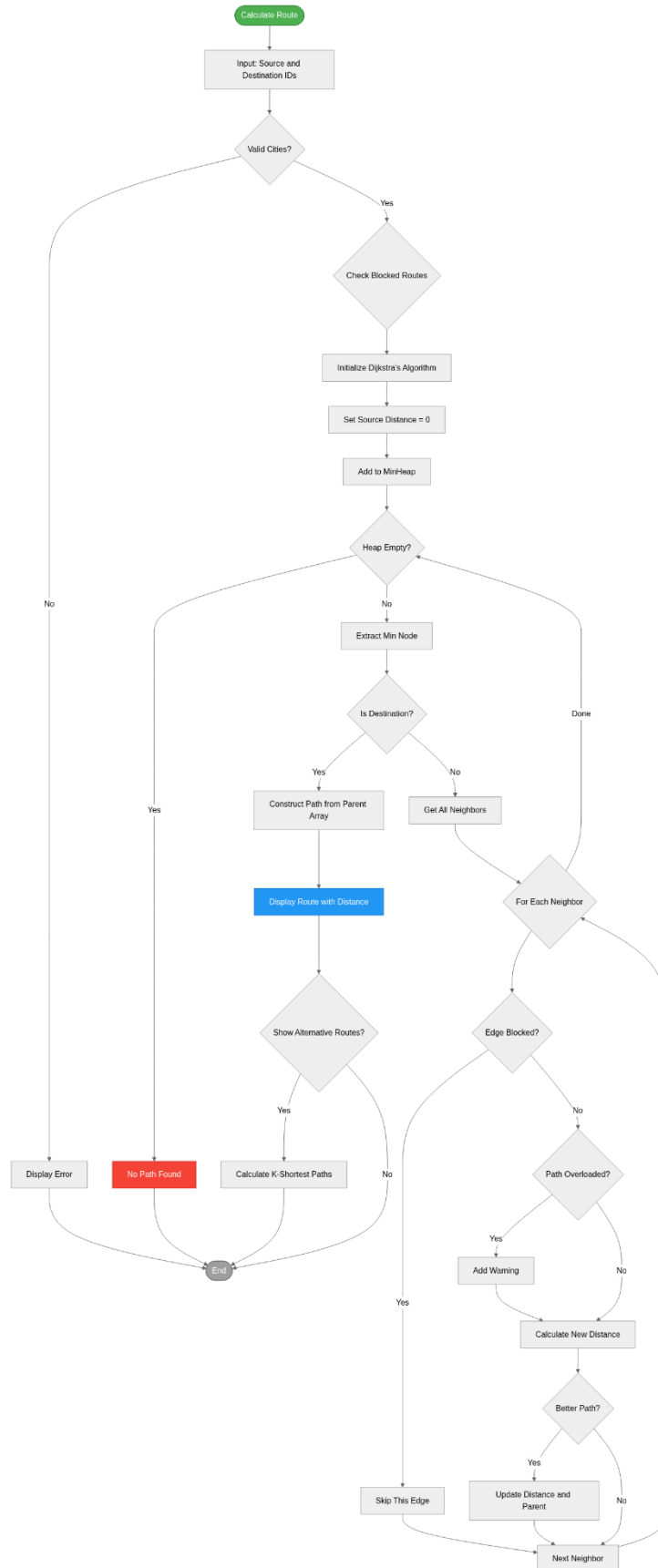
Operation Menu



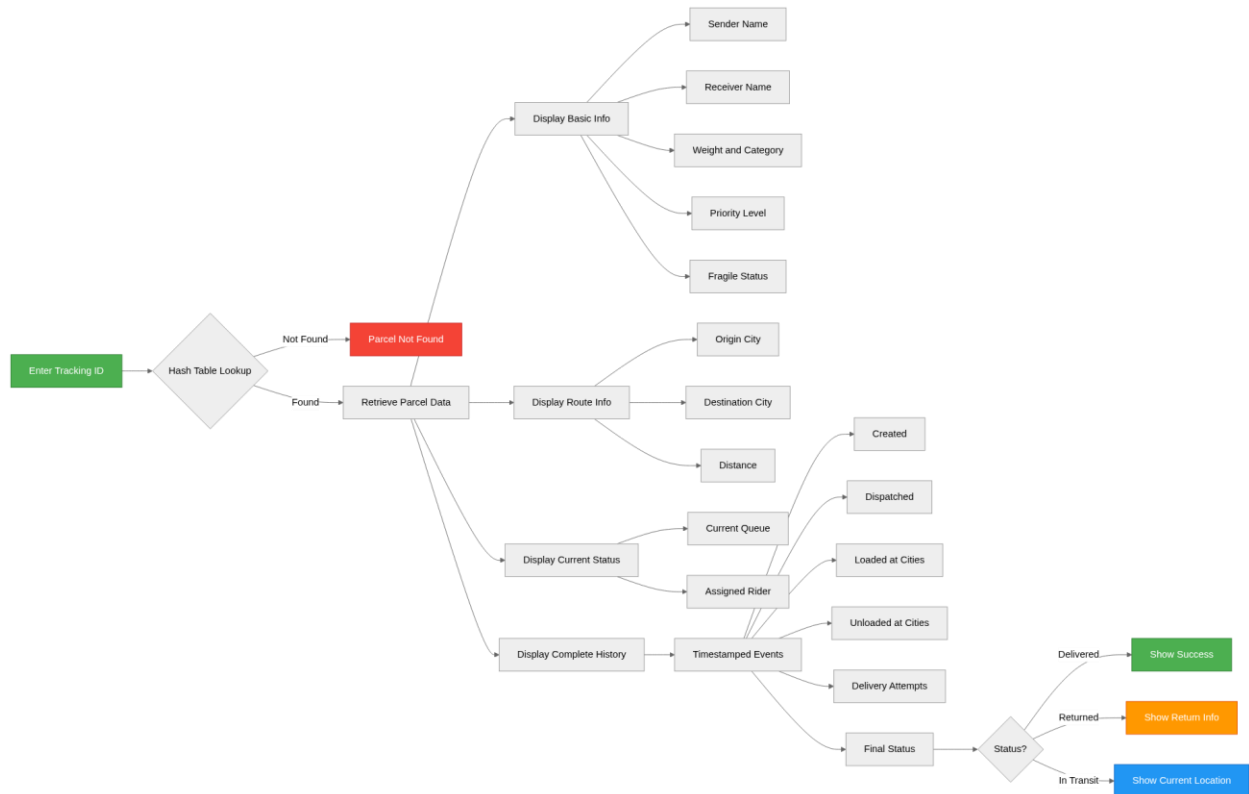
Parcel Lifecycle



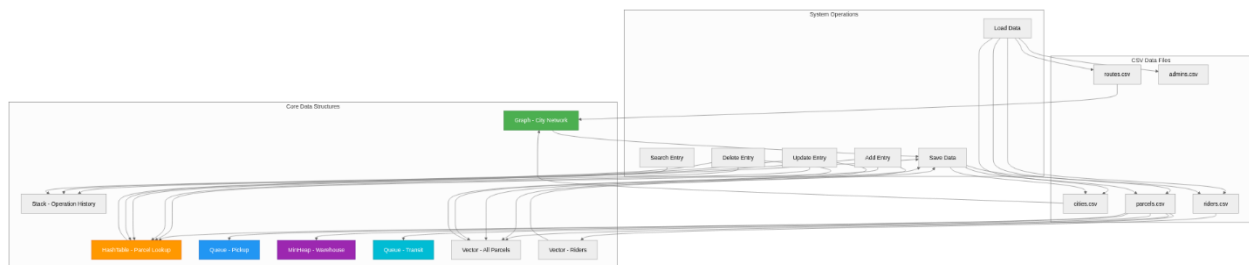
Routing



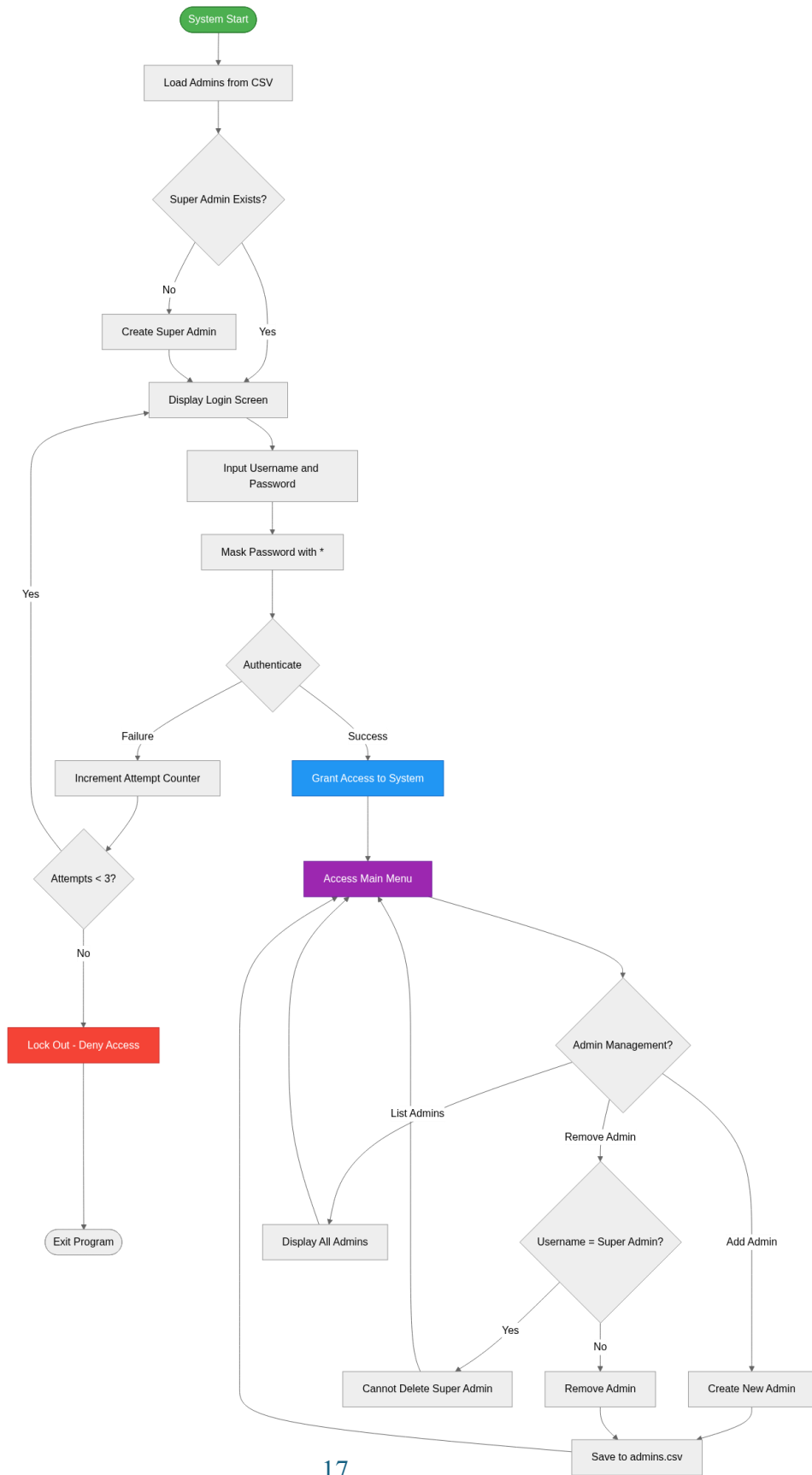
Parcel Tracking



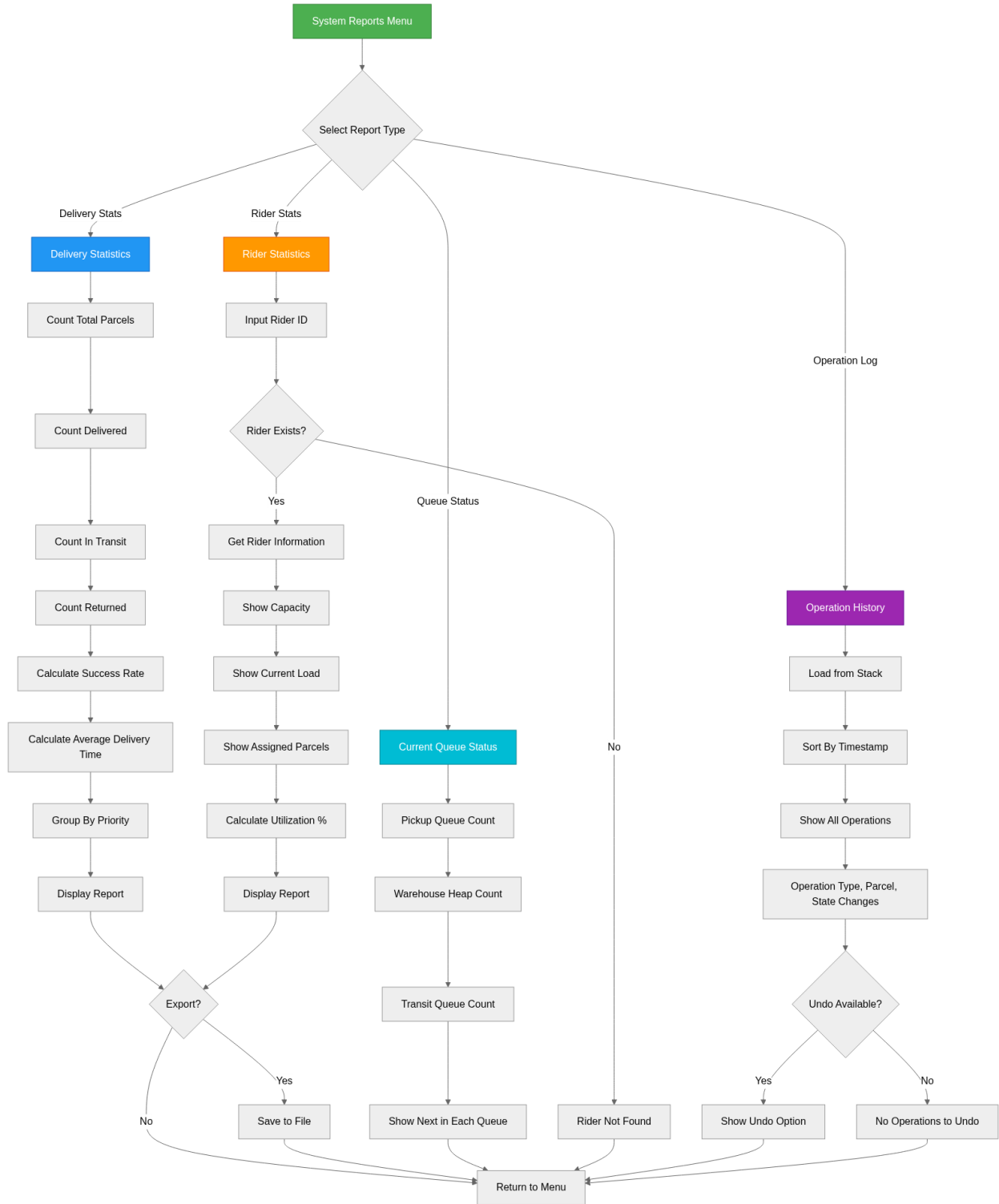
Data Structures



Authentication



Report Statistics



Conclusion

The SwiftEx Courier System successfully implements an intelligent logistics engine using custom data structures and algorithms. The system demonstrates:

1. **Effective Data Structure Usage:** 7+ custom data structures used appropriately
2. **Algorithm Efficiency:** Optimized algorithms with complexity analysis
3. **Modular Design:** Clear separation of concerns and modular architecture
4. **Comprehensive Functionality:** All required modules fully implemented
5. **Code Quality:** Well-documented, readable, and maintainable code

Future Enhancements

Potential improvements for production systems: 1. Database integration instead of CSV files 2. Real-time GPS tracking integration 3. Web-based user interface 4. Multi-threading for concurrent operations 5. Machine learning for route optimization 6. Mobile app for riders and customers

Learning Outcomes

This project successfully demonstrates:

CLO 3: Design and develop solutions using suitable data structures

CLO 4: Comply with ethics and professional practices in computing

The implementation showcases deep understanding of data structures, algorithms, and software engineering principles, meeting all project requirements and learning objectives.

References

1. Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to Algorithms* (3rd ed.). MIT Press.
2. Data Structures and Algorithms - UET Lahore Course Materials
3. Dijkstra, E. W. (1959). A note on two problems in connexion with graphs. *Numerische Mathematik*, 1(1), 269-271.
4. Yen, J. Y. (1971). Finding the k shortest loopless paths in a network. *Management Science*, 17(11), 712-716.

Project Developed By: Arslan & Co.

Date: December 2024

Version: 1.0